
jerrypi

把 Pi coding agent 装进 VS Code 侧边栏 —— 不装 Node、不跑脚本，装完扩展就能用

What's in here

- 01 为什么做 不折腾配置，装完即用
- 02 Pi 的优势 简单 · 可控 · 好懂 · 可扩展
- 03 经验教训 文档先行 · 状态明确 · 多 review · 规范验收
- 04 推荐尝试 三件能马上上手的事

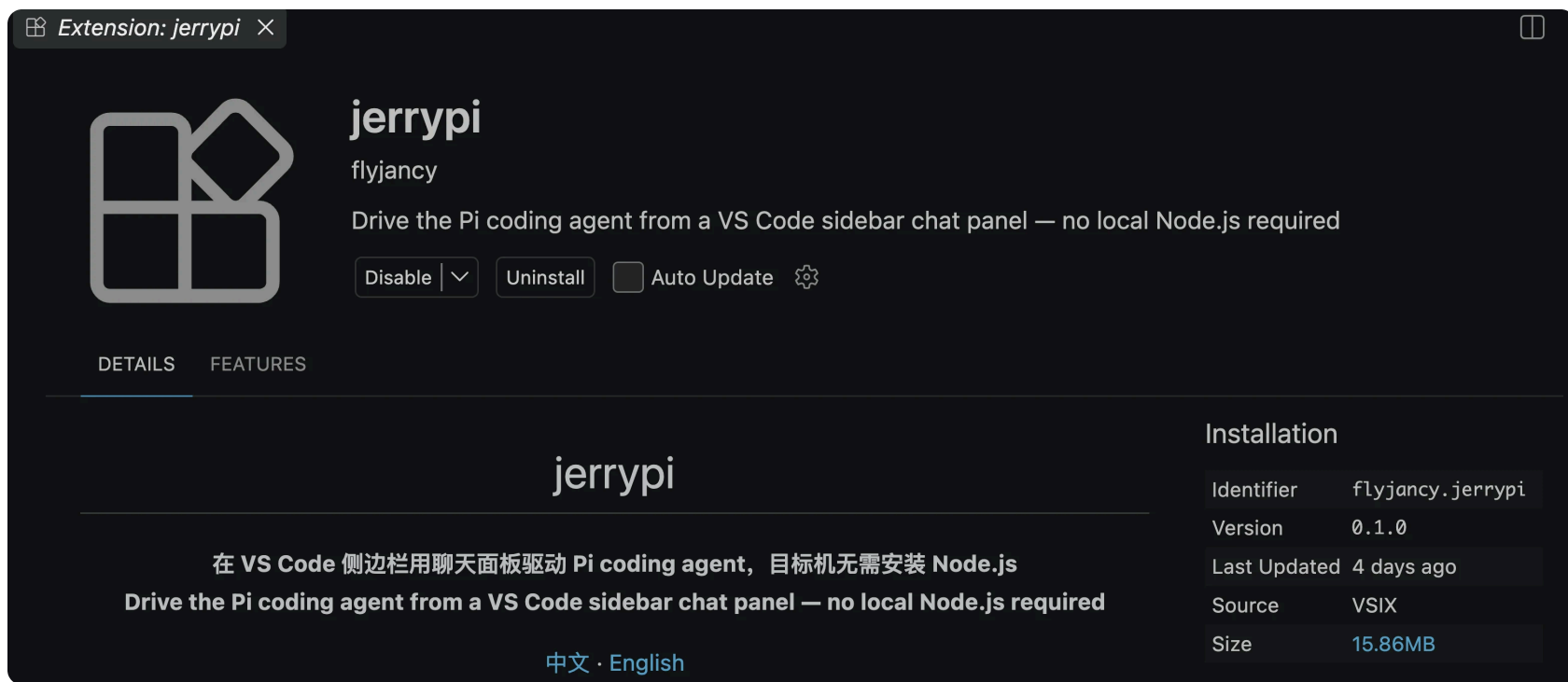
One extension is the whole setup

起因：让「不想折腾配置」的人，也能直接用上 Pi coding agent。

- 01 **不装环境** 目标机不用装 Node —— 扩展宿主自带 Node 24
- 02 **开箱即用** pi 的 SDK bundle 随扩展分发，面板与工具卡片装完就有
- 03 **一份配置** 沿用 pi 的 ~/.pi/agent 配置：同一份 models.json，不用重来

把「装环境」这件事，压缩成装一个扩展。

One VSIX, nothing else



Why Pi: simple, controllable, hackable

A 简单

一个 npm 包、零原生依赖；配置就是 ~/.pi/agent 下几个 JSON 文件。

B 可控

模型、工具、审批门槛全是自己的设置；会话是纯 JSONL，可审计可迁移。

C 好懂

文档与源码齐全，行为可预测 —— 出问题能顺着查到根因。

D 可扩展

TypeScript 扩展、SDK、主题；这个 VS Code 扩展本身就是用 SDK 拼出来的。

能用 SDK 拼出一个 IDE 扩展，就是可扩展性最好的证明。

Write it down before you build it

- 01 **文档先行** 目标、判据、阶段先写成文档，再动手写代码
- 02 **单一来源** 同一件事只留一份文档 —— 复制出来的副本一定会烂掉
- 03 **状态明确** 每次提交前更新「现在在哪 / 欠什么 / 下一步」

agent 的上下文有限 —— 只有写进文件的东西留得住。

Get it reviewed, then prove it

- 01 **多轮 review**：同一个模型看不出自己的盲区 —— 换一个更强的模型当评审。
- 02 **评审意见先复现再采纳**：把它当成一条新断言跑一遍，绿不了就不采纳。
- 03 **规范验收**：先写断言、看红、再实现；SKIP 不是 PASS；人工验收固定四类。

每次改动自动跑 329 条 host-check 与 16 项 gating self-test；验收挂了就再来一轮。

假绿比红贵 —— 它让你以为已经验证过了。

Three ways to poke at it

- 01 **装扩展**
VSIX 仓库里的 jerrypi-0.1.12.vsix —— Install from VSIX
- 02 **写扩展**
EXTEND 抄 pi-config 的 footer 扩展，几十行加一条命令
- 03 **换模型**
MODELS models.json 里加自定义 provider，面板里就能选

Thanks